Práctica: Integración de herramientas MCP en EBIS-CLI

Contexto

En la práctica 1 completaste EBIS-CLI, un agente que conserva el historial de la sesión y puede ejecutar tools locales para trabajar con archivos y hacer cálculos. En esta práctica conectarás ese agente con un servidor externo mediante el Model Context Protocol (MCP).
El servidor expondrá operaciones de EBIS Calendar API como tools MCP. EBIS-CLI descubrirá esas tools al arrancar, las añadirá a las tools locales que ya conoce el modelo y enviará al servidor las llamadas que correspondan. El servidor MCP, a su vez, utilizará el cliente HTTP que se entrega para consultar o modificar el calendario.

En esta práctica, EBIS-CLI arrancará Calendar MCP mediante stdio: el cliente inicia el servidor como un proceso hijo y se comunica con él por sus entradas y salidas estándar. Así no hay que abrir otro puerto para la conexión MCP. La API REST de calendario sí seguirá escuchando en *http://127.0.0.1:8000*; son dos conexiones distintas.

El directorio *practica_2* ya contiene las tres carpetas de trabajo. La copia de *ebis-cli-proyect* parte de la solución de la práctica 1 y sigue la estructura de *ebis-cli-mcp*; se han dejado TODO únicamente en los puntos de integración que completarás. En *ebis-calendar-api-mcp*, *client.py* está terminado y *server.py* tiene huecos para registrar y ejecutar las tools.
Completarás los TODO de *ebis_cli/mcp.py*, *ebis_cli/agent.py* y *ebis_calendar_api_mcp/server.py*. La estructura asíncrona, el bucle principal del agente, la configuración, la interfaz, el cierre de la conexión y las funciones auxiliares de conversión se entregan preparadas. En los métodos asíncronos ya escritos, completa las llamadas indicadas a la SDK MCP y conserva las instrucciones *await* que aparecen en el esqueleto.

Objetivos

Al terminar la práctica serás capaz de:

- Crear un cliente MCP que inicie el servidor de calendario mediante stdio.
- Conectar con un servidor MCP, descubrir sus tools e invocarlas.
- Convertir las tools descubiertas al formato de funciones que recibe el modelo.
- Combinar las tools MCP con las tools locales de EBIS-CLI.
- Enrutar cada llamada del modelo al dispatcher local o al servidor MCP.
- Exponer operaciones de una API existente como tools de un servidor MCP.
- Entender por qué el servidor MCP de calendario se inicia como un proceso stdio.

Material inicial

Trabajarás dentro de la carpeta *practica_2*, que contiene estos proyectos:

```
practica_2/
├── ebis-cli-proyect/       # Solución de la práctica 1, preparada para integrar MCP
├── ebis-calendar-api/      # API REST de calendario, completa
└── ebis-calendar-api-mcp/  # Cliente HTTP y esqueleto de server.py
```

En *ebis-cli-proyect* se conservan el agente y las tools locales de la práctica 1. Ya están incluidos *mcp_config.py* para leer la configuración, *mcp_tools.py* para adaptar herramientas y resultados, los archivos JSON de configuración y los puntos de arranque y cierre de la interfaz. Completarás solo los TODO de *ebis_cli/mcp.py* y *ebis_cli/agent.py*; no necesitas modificar la TUI, el cierre ni las tools locales.

En *ebis-calendar-api-mcp*, *client.py* contiene las peticiones HTTP a la API y *server.py* conserva la estructura del servidor con TODO en la inicialización de MCPServer y en las funciones de tools. Completarás esos huecos. No necesitas cambiar la lógica de *ebis-calendar-api/api.py* ni la de *ebis_calendar_api_mcp/client.py*.

La API guarda sus datos en memoria. Al reiniciarla, las reuniones creadas durante la práctica desaparecen. La lista de reuniones iniciales se genera para el siguiente día laborable.

Preparación del entorno

Los tres proyectos usan sus propias dependencias. Ejecuta *uv sync* desde cada carpeta de proyecto la primera vez que trabajes con ella. Conserva las versiones MCP declaradas en cada *pyproject.toml*: el cliente y el servidor se ejecutan en entornos distintos y se comunican mediante el protocolo MCP.

Configurar EBIS-CLI

Desde *ebis-cli-proyect*, prepara el entorno:

```
uv sync
```

Copia *.env.example* como *.env* y añade tu clave en *OPENAI_API_KEY*. Puedes conservar el modelo configurado en *OPENAI_MODEL* o elegir uno disponible para tu cuenta. No compartas ni subas *.env*. En Linux:

```
cp .env.example .env
```

El proyecto cliente ya declara *mcp>=2.0.0,<3.0.0* en su *pyproject.toml*. Ejecuta *uv sync* para instalarlo.

Preparar la API de calendario

Abre una terminal en *ebis-calendar-api* y ejecuta:

```
uv sync
uv run python main.py
```

La API estará disponible en *http://127.0.0.1:8000*. Puedes comprobar su documentación interactiva en *http://127.0.0.1:8000/docs*.

Preparar el servidor MCP

Desde *ebis-calendar-api-mcp*, instala sus dependencias:

```
uv sync
```

En el modo stdio predeterminado, EBIS-CLI iniciará el servidor como un proceso hijo; no hace falta abrir una terminal para el servidor MCP. El proceso del servidor se conectará a la API en *http://127.0.0.1:8000*.

Cómo se conectan las piezas

El recorrido de una llamada MCP tiene estos pasos:

1. EBIS-CLI crea un cliente MCP usando la configuración del servidor.
2. El cliente abre la conexión y solicita la lista de tools disponibles.
3. El agente añade los esquemas MCP a los esquemas de sus tools locales y los envía al modelo.
4. Si el modelo elige una tool local, EBIS-CLI la ejecuta con *dispatch_tool*. Si elige una tool MCP, EBIS-CLI la solicita al servidor conectado.
5. El resultado vuelve al agente como salida de la llamada de función. El modelo puede usarlo para preparar su respuesta o decidir el siguiente paso.
6. Al cerrar la aplicación, EBIS-CLI cierra la conexión MCP y el proceso stdio asociado.

El módulo *mcp_tools.py* ya convierte los esquemas recibidos desde MCP al formato de funciones de Responses API. También antepone *mcp_* a los nombres que presenta al modelo y conserva un mapa para traducirlos a los nombres originales del servidor. Por ejemplo, *list_workers* se presenta como *mcp_list_workers*.

Parte 1: completar el cliente MCP

Abre *ebis_cli/mcp.py*. El esqueleto ya contiene *MCPManager*, sus métodos asíncronos y el manejo de errores. Reemplaza las expresiones *TODO* indicadas por las llamadas a la SDK MCP. No reescribas los métodos ni cambies su estructura.

1. Crear el cliente stdio en *_create_client*

El parámetro *server* contiene los datos validados desde el archivo JSON de configuración. La rama HTTP de *_create_client* ya está implementada como alternativa. En esta práctica, *mcp_config.json* indica *transport: "stdio"*, así que completa la rama que crea el proceso MCP:

- Crea *StdioServerParameters* con *command*, *args*, *env* y *cwd* de *server*.
- Pasa ese objeto a *Client* y devuelve el cliente.

Completa *TODO_STDIO_PARAMETERS*. No cambies los nombres de las claves: *mcp_config.py* las valida y las prepara antes de llamar a esta función. El cliente MCP utilizará *command* y *args* para lanzar Calendar MCP, *env* para pasarle la URL de la API y *cwd* como carpeta inicial del proceso.

2. Abrir la conexión y descubrir tools

En *_start*, el esqueleto crea el cliente y mantiene la conexión dentro del ciclo de vida de *MCPManager*. Completa las llamadas MCP indicadas para:

- Abrir el cliente con *await client.__aenter__()*.
- Obtener la lista de tools con *await client.list_tools()*.
- Pasar *result.tools* a *adapt_tools*.
- Guardar los esquemas que devuelve *adapt_tools* y el mapa de nombres públicos a nombres MCP.

Reemplaza *TODO_ENTER*, *TODO_LIST_TOOLS* y *TODO_TOOLS* por esas llamadas y el atributo correspondiente. Las firmas *async def*, los *await*, el bloque de control de errores y el cierre en caso de fallo ya están escritos en el esqueleto.

3. Ejecutar una tool MCP

En *call_tool*, utiliza el mapa *model_tool_name_to_mcp_name* para encontrar el nombre que reconoce el servidor. Después, reemplaza *TODO_CALL_TOOL* para llamar a *self.client.call_tool(...)* con ese nombre y el diccionario *arguments* recibido. El método *format_tool_result* ya transforma el resultado MCP en texto para el agente y se ocupa de los errores comunicados por el servidor.
Si no hay una conexión activa o el nombre no aparece en el mapa, conserva los mensajes de error preparados en el esqueleto.
El resto de *MCPManager*, incluido su método *close*, ya está programado. No modifiques ese ciclo de vida. Completa las llamadas MCP señaladas y conserva la estructura asíncrona proporcionada.

Parte 2: conectar MCP con el agente

Abre *ebis_cli/agent.py*. El agente conserva las tools locales de la práctica 1 y su loop de Responses API. Completa los cinco TODO de integración.

1. Crear y arrancar el gestor MCP

En *Agent.__init__*, completa el TODO 1 inicializando una instancia de *MCPManager* y guardándola en *self.mcp*.
Completa el TODO 2 de *start_mcp* para esperar a *self.mcp.start()* y devolver el mensaje de estado. La interfaz ya llama a *start_mcp* al iniciar. Ese método lee la configuración, abre la conexión y carga los esquemas disponibles. El método *close* y su llamada al salir ya están implementados.

2. Combinar las tools locales y MCP

Completa el TODO 3 de *available_tool_schemas* para devolver una sola lista con:

- *TOOLS*, que contiene las funciones locales de EBIS-CLI.
- *self.mcp.tools_for_model*, que contiene las tools descubiertas en el servidor.

El método *available_tools* ya obtiene los nombres desde *available_tool_schemas*. Al completar el TODO 3, */tools* mostrará tanto las tools locales como las remotas.
En la llamada a *self.client.responses.create(...)*, reemplaza el TODO 4 para pasar el resultado de *available_tool_schemas()* como *tools*. El modelo necesita recibir todos los esquemas en la misma petición para poder elegir entre una tool local y una MCP.

3. Enrutar las llamadas

El loop ya detecta las llamadas a funciones del modelo y agrega sus resultados a la conversación. Completa la selección del ejecutor:

- Reemplaza el TODO 5 con una comprobación de si *call.name* aparece en *self.mcp.model_tool_name_to_mcp_name*. Si aparece, ejecuta *await self.mcp.call_tool(call.name, arguments)*.
- En cualquier otro caso, ejecuta la función local mediante *dispatch_tool(call.name, arguments)*.

Conserva el nombre público *call.name* al consultar el mapa. El gestor MCP se ocupa de traducirlo al nombre original del servidor.

Parte 3: completar el servidor MCP de calendario

Abre *ebis_calendar_api_mcp/server.py*. El servidor debe exponer las operaciones del cliente HTTP como tools MCP. Los decoradores *@mcp.tool()*, las firmas y los docstrings ya están escritos. MCPServer utiliza las anotaciones de tipo y docstrings para describir las tools al cliente. Completa los TODO sin cambiar las firmas.
Implementa estas tres tools delegando en las funciones de *ebis_calendar_api_mcp.client*:

Tool MCP	Argumentos	Operación del cliente HTTP
`list_workers`	Ninguno	`client.list_workers()`

Tool MCP	Argumentos	Operación del cliente HTTP
`get_worker_meetings`	`worker_id: int, day: str | None = None`	`client.get_worker_meetings( worker_id=..., day=...)`
`schedule_meeting`	`worker_id: int, title: str, start: str, end: str`	`client.schedule_meeting(...)`

La API recibe fechas como texto ISO local, por ejemplo *2026-10-01T14:00:00*, sin zona horaria. La API comprueba el trabajador, el horario de lunes a viernes entre las 09:00 y las 17:00, que el final sea posterior al inicio y que no exista un solapamiento. El servidor MCP debe llamar al cliente HTTP y devolver su resultado; no dupliques esas validaciones.

Transporte usado en esta práctica

El servidor lee *MCP_TRANSPORT*. Si no defines esa variable, usa *stdio*, que también es el valor predeterminado. En *server.py*, la rama HTTP ya está completa como alternativa ilustrativa; no necesitas cambiarla ni arrancar el servidor MCP manualmente.

Completa el TODO 1 para crear la instancia *MCPServer* del modo stdio. La función *main* ya llama a *mcp.run()* para este transporte. Completa los TODO 2, 3 y 4 para que las tres tools deleguen en la operación equivalente de *client.py* y devuelvan su resultado. El recurso *ebis-calendar://config* y el resto de la función *main* ya están preparados.

Cuando uses stdio, no escribas mensajes de depuración en la salida estándar: el protocolo MCP utiliza ese canal para comunicarse con el cliente.

Configuración del cliente

mcp_config.json está en la raíz del proyecto cliente. *mcp_config.py* lo lee, valida sus campos y convierte la carpeta de trabajo en una ruta absoluta. El agente usa estos datos para construir *StdioServerParameters*; la SDK MCP inicia el servidor hijo con ellos.

Este es el archivo que se entrega:

```
{
  "enabled": true,
  "server": {
    "name": "ebis-calendar-api-mcp",
    "transport": "stdio",
    "command": "uv",
    "args": [
      "--directory",
      "../ebis-calendar-api-mcp",
      "run",
      "python",
      "-m",
      "ebis_calendar_api_mcp.server"
```

```
      ],
      "env": {
        "EBIS_CALENDAR_API_URL": "http://127.0.0.1:8000"
      }
    }
}
```

Cada bloque responde a una pregunta distinta sobre cómo debe encontrar y arrancar el servidor:

1. *enabled: true* permite que el agente intente conectar con el servidor configurado. Si se cambia a *false*, EBIS-CLI conserva únicamente sus tools locales.
2. *server* reúne los datos de un servidor MCP. *name* es una etiqueta que el agente muestra en su mensaje de estado; no es un nombre de paquete de Python.
3. *transport: "stdio"* elige la comunicación por entrada y salida estándar. En este modo no hace falta reservar un puerto para MCP: EBIS-CLI inicia el servidor como proceso hijo y ambos procesos intercambian mensajes por sus canales estándar.
4. *command: "uv"* indica qué programa ejecutará el cliente. *args* es una lista porque el proceso recibe argumentos separados; cada elemento del JSON equivale a un argumento de la línea de comandos, en el mismo orden.
5. *--directory* seguido de *../ebis-calendar-api-mcp* indica a *uv* dónde está el proyecto del servidor: es una carpeta hermana de *ebis-cli-proyect*. Así *uv run* usa las dependencias del *pyproject.toml* del servidor, aunque el comando lo haya iniciado el cliente.
6. *run* pide a *uv* ejecutar el comando siguiente en ese entorno. *python -m ebis_calendar_api_mcp.server* ejecuta el módulo *server* del paquete *ebis_calendar_api_mcp*; al final de ese módulo, *main()* inicia MCPServer.
7. *env* define variables de entorno para el proceso hijo. *EBIS_CALENDAR_API_URL* indica al cliente HTTP del servidor MCP dónde está EBIS Calendar API. La API REST escucha en el puerto *8000*; ese puerto pertenece a la API, no al transporte stdio de MCP.

mcp_config.py usa la carpeta del propio archivo de configuración como directorio de trabajo del proceso. Desde allí, *../ebis-calendar-api-mcp* apunta al proyecto MCP.

También se entrega *mcp_config.http.json* como comparación. Ese archivo usa *transport: "http"* y la URL *http://127.0.0.1:8001/mcp*; apunta a un servidor que ya está levantado, por lo que no necesita *command*, *args* ni *env*. Se conserva para mostrar la diferencia entre ambos transportes. La práctica y sus comprobaciones usan *mcp_config.json* con stdio.

Comprobación guiada

1. Probar stdio

Abre una terminal para la API REST. Desde la carpeta que contiene *practica_2*, ejecuta:

```
cd practica_2/ebis-calendar-api
uv sync
uv run python main.py
```

La API debe quedar activa en *http://127.0.0.1:8000*. Abre otra terminal, prepara EBIS-CLI y arráncalo:

```
cd practica_2/ebis-cli-proyect
uv sync
cp .env.example .env
# Edita .env y añade OPENAI_API_KEY.
uv run python -m ebis_cli.app
```

La interfaz inicia Calendar MCP automáticamente por stdio usando *mcp_config.json*; no abras otra terminal ni ejecutes *server.py* manualmente. Al arrancar, el agente debe indicar que se conectó al servidor y cuántas tools descubrió. Escribe */tools* y confirma que aparecen las cinco tools locales de la práctica 1 y estas tres tools MCP:

```
mcp_list_workers
mcp_get_worker_meetings
mcp_schedule_meeting
```

Prueba una tool de calendario:

```
Usa mcp_list_workers para mostrar los trabajadores disponibles y sus horarios laborales.
```

La respuesta debe incluir a Ana García, Luis Martín y Marta López. Después, comprueba las reuniones iniciales:

```
Usa mcp_get_worker_meetings para consultar las reuniones del trabajador 1.
```

La API devuelve las reuniones con sus fechas y horas. Por último, agenda una reunión de prueba para el trabajador 3 en un día laborable futuro, entre las 09:00 y las 17:00. Vuelve a consultar sus reuniones y confirma que aparece la reunión creada.

Comprueba también que siguen funcionando las tools locales. Por ejemplo, pide al agente que use *count_words* con una frase breve o *list_files* para enumerar archivos.

Problemas frecuentes

- **No aparece ninguna tool MCP:** confirma que la API REST sigue activa, que *mcp_config.json* apunta a *../ebis-calendar-api-mcp* y que *uv sync* se ejecutó en el proyecto del servidor.
- **Error al importar *mcp*:** ejecuta *uv sync* en la carpeta del proyecto que muestra el error. Cada proyecto mantiene su propio entorno.
- **El modelo anuncia una tool MCP, pero no se ejecuta:** comprueba que el nombre público conserva el prefijo *mcp_*, que se consulta *model_tool_name_to_mcp_name* y que el dispatcher MCP recibe el nombre y los argumentos.
- **La tool responde con un error de calendario:** revisa la fecha, el horario laboral, el identificador del trabajador y los posibles solapamientos. La respuesta de error de la API también llega al agente como resultado de la tool.
- **El proceso stdio parece no hacer nada:** no abras el servidor MCP en otra terminal. EBIS-CLI lo inicia al arrancar. Revisa que la API REST esté activa y consulta */tools* en la interfaz.

Referencias

- SDK oficial de MCP para Python
- Cliente de la SDK MCP para Python
- Tools con MCPServer
- Ejecución de servidores MCP en la SDK para Python
- Documentación de EBIS Calendar API: *http://127.0.0.1:8000/docs*